1. Identities equivalent to $\operatorname{atan}(1)$

A demonstration that

$$8\operatorname{atan}\tfrac{1}{10} - \operatorname{atan}\tfrac{1}{239} - 4\operatorname{atan}\tfrac{1}{515} = \operatorname{atan} 1 \tag{1.1}$$

First, recall the sum-of-angles identity for the tangent function:

$$\tan(x+y) = \frac{\tan x + \tan y}{1 - (\tan x)(\tan y)} \tag{1.2}$$

Furthermore, by the nature of inverse functions,

$$x = \operatorname{atan}(\tan x) = \tan(\operatorname{atan} x) \tag{1.3}$$

Therefore:

$$\begin{aligned} \operatorname{atan} x + \operatorname{atan} y &= \operatorname{atan}(\tan(\operatorname{atan} x + \operatorname{atan} y)) \\ &= \operatorname{atan}\frac{\tan(\operatorname{atan} x) + \tan(\operatorname{atan} y)}{1 - \tan(\operatorname{atan} x)\tan(\operatorname{atan} y)} \\ &= \operatorname{atan}\frac{x+y}{1-xy} \end{aligned} \tag{1.4}$$

And, as a special case,

$$\begin{aligned} 2\operatorname{atan} x &= \operatorname{atan} x + \operatorname{atan} x \\ &= \operatorname{atan}\frac{2x}{1-x^2} \end{aligned} \tag{1.5}$$

Thus:

$$\begin{aligned} &8\operatorname{atan}\tfrac{1}{10} \\ &= 4\operatorname{atan}\tfrac{2/10}{1-1/100} && = 4\operatorname{atan}\tfrac{20}{99} \\ &= 2\operatorname{atan}\tfrac{40/99}{1-400/9801} && = 2\operatorname{atan}\tfrac{3960}{9401} \\ &= \operatorname{atan}\tfrac{7920/9401}{1-15681600/88378801} && = \operatorname{atan}\tfrac{74455920}{72697201} \end{aligned} \tag{1.6}$$

And

$$\begin{aligned} &4\operatorname{atan}\tfrac{1}{515} \\ &= 2\operatorname{atan}\tfrac{515}{132612} \\ &= \operatorname{atan}\tfrac{136590360}{17585677319} \end{aligned} \tag{1.7}$$

So

$$\begin{aligned}
&8\,\mathrm{atan}\,\tfrac{1}{10} - \mathrm{atan}\,\tfrac{1}{239} - 4\,\mathrm{atan}\,\tfrac{1}{515} \\
&\quad = \mathrm{atan}\,\tfrac{74455920}{72697201} - (\mathrm{atan}\,\tfrac{1}{239} + \mathrm{atan}\,\tfrac{136590360}{17585677319}) \\
&\quad = \mathrm{atan}\,\tfrac{74455920}{72697201} + \mathrm{atan}\,\tfrac{-1758719}{147153121} \\
&\quad = \mathrm{atan}\,1 \quad \blacksquare
\end{aligned}$$

Another identity that could be used is:

$$\mathrm{atan}\,1 = 4\,\mathrm{atan}\,\tfrac{1}{5} - \mathrm{atan}\,\tfrac{1}{239} \tag{1.8}$$

This has the advantage of only requiring two passes, but has the disadvantage that the series for $\frac{1}{5}$ converges more slowly than those for $\frac{1}{10}$ and $\frac{1}{515}$ combined. The proof of correctness may be obtained in a manner very similar to the one shown above, and will not be spelled out here.

[Note that these Taylor-series based computations are not state-of-the-art for computing absurd quantities of digits of tau or pi; the trillion-plus digit record holders use a hypergeometric series developed by the brothers David and Gregory Chudnovsky, which cranks out about 15 digits per term computed for the series. Another approach involves refinements of the Gauss-Legendre algorithm, such as one by Richard Brent and Eugene Salamin (1976), which *doubles* the number of accurate digits with each iteration. But these approaches are not as easy to understand or implement as Taylor-series based approaches like the one used in this program.]

Putting this all together and running calculations using multi-precision arithmetic with "digit"s of base `BASE` is how this code accomplishes its task.

2. Error Analysis

Let

$$f(x,n) = \frac{x^{2n+1}}{2n+1} \quad [1 \le \tfrac{1}{x};\ \text{i.e., } 0 \le x \le 1] \tag{2.1}$$

Then the Taylor-Maclaurin series for $\operatorname{atan} x$ can be written as:

$$\operatorname{atan} x = \sum_{i=0}^{\infty} (-1)^i\, f(x,i) \tag{2.2}$$

Recall that if a_i is a monotonically non-increasing sequence with $\lim_{i\to\infty} a_i = 0$, then the sum $\sum_{i=0}^{\infty}(-1)^i\, a_i$ converges; truncating this sum after term n will have an error whose magnitude is at most a_{n+1}. Thus for $\operatorname{atan} x$, provided that $1 \le \frac{1}{x}$, the error incurred by truncating the series after the n-th term is:

$$\text{err}(n) \le f(x,n+1) = \frac{x^{2n+3}}{2n+3} < \frac{x^{2n+1}}{2n+1} = f(x,n) \tag{2.3}$$

We will assume that $a = 1/x$ is the worst-case choice for the relations being considered. Substituting into (2.3) gives

$$\text{err}(n) < f(\tfrac{1}{a}, n) \tag{2.4}$$

We want our error term $\text{err}(n)$ to be less than B^{-D}, for some specified base B and to-be-determined base-B-digit-count D:

$$\text{err}(n) < B^{-D} \tag{2.5}$$

Furthermore, we want to fit intermediate results into an integer which cannot exceed some maximum $I_{\max}$. Each word of our multi-precision vector is modulo-reduced by $2n+1$ on outer-loop iteration n. Each step computes

$$r_i B + t_i \le I_{\max}$$

at word-index i, for carry-down residue r_i and previous-term t_i, each of which is in the range $0 \le \star \le 2n+1-1 = 2n$, thus the worst-case intermediate result is $2\,nB + 2\,n \le I_{\max}$. Solving for n yields:

$$n \le \frac{I_{\max}}{2\,(B+1)} \tag{2.6}$$

Our goal is to find the largest D that simultaneously satisfies relations (2.4), (2.5), and (2.6). It is acceptable to find a value D' that is less than the actual maximum, if an approximation makes life easier, but we must never allow the resultant D' to be bigger than D and we prefer not to underestimate D by *too* much.

Now equations (2.4) and (2.5) do not have a natural ordering, but given our goal of wanting to derive a "safe" value of D, we choose to combine them with this ordering:

$$\mathrm{err}(n) < f(\tfrac{1}{a}, n) < B^{-D} \tag{2.7}$$

For brevity, let

$$R = \frac{I_{\max}}{B+1} \tag{2.8}$$

Combining substitutions of (2.6) and (2.8) into equation (2.7) gives:

$$f(\tfrac{1}{a}, \tfrac{R}{2}) < B^{-D}$$

Solving for D yields:

$$D < \frac{-\log f(\tfrac{1}{a}, \tfrac{R}{2})}{\log B} \tag{2.9}$$

Using $(\frac{1}{a})^x = a^{-x}$, $\log x / \log B = \log_B x$, and the definition of f given in equation (2.1):

$$D < \frac{-\log \frac{a^{-(2(R/2)+1)}}{2(R/2)+1}}{\log B} = (R+1)\log_B a - \log_B(R+1) \tag{2.10}$$

Converting the digit count to base-10 digits involves multiplying by $\log_{10} B$. Because

$$(\log_B x)(\log_{10} B) = \frac{\log_B x}{\log_B 10} = \log_{10} x$$

we can determine that the constraint on the number of decimal digits d as

$$d = D\log_{10} B < (R+1)\log_{10} a - \log_{10}(R+1) \tag{2.11}$$

To take a concrete example, let us assume that our worst-case is $a = 5$ (as when evaluating the relation in equation (1.8)), that $B = 10^{12}$, and that $I_{\max} = 2^{63} - 1$; then $R = \frac{2^{63}-1}{10^{12}+1}$ and we get

$$d < \left(\tfrac{2^{63}-1}{10^{12}+1} + 1\right)\log_{10} 5 - \log_{10}\left(\tfrac{2^{63}-1}{10^{12}+1} + 1\right) \approx 6\,446\,844$$

So for these parameters we want to ensure that no more than 6 446 844 decimal digits are allowed to be computed and printed, in order to avoid the risk of errors from integer overflow (whether they are silent as in C or panicking as in Rust).

Some further adjustments:

- To re-iterate, we don't need to be precise in our upper bound; lowering by a few digits is acceptable.

- Hence it is okay to increase the $\log_{10}(R+1)$ term in equation (2.11) by a little bit, as this would lower the bound on d by a little bit. The largest $I_{\max}$ supported by the code (due to its use of libdivide, which only supports 32- and 64- bit integers) is $2^{63}-1$, and $\log_{10} 2^{63} < 20$, so we can safely substitute 20 for this term, regardless of what the actual $I_{\max}$ and B used might be.
- We are assuming that our worst case is $a = 5$, so $\log_{10} a > 0.698$.
- To minimize user confusion, we want to round the "maximum" down to the nearest multiple of $\log_{10} B$, since we will always be outputting an integer number of base-B "digits".

All of this can be accomplished with integer-only calculations by using the following Rust function. Here `Xword` is the signed-integer type used for calculations, `Xword::MAX` represents $I_{\max}$, `WORDDIGITS` $= \log_{10} B$; we require that `WORDDIGITS` be an integer, hence that we end up with `BASE` $= B$ below:

```
const BASE: Xword = (10 as Xword).pow(WORDDIGITS as u32);
fn max_digits() -> Xword {
  let d = (Xword::MAX / (BASE+1) + 1) * 698 / 1000 - 20;
  d - d % (WORDDIGITS as Xword)
}
```

Note that if we use $a = 10$ (like if we use equation (1.1)) instead of $a = 5$ (as with equation (1.8)), then instead of using $\log_{10} 5 \approx 698/1000$ we would use $\log_{10} 10 = 1$, which both simplifies the equation and allows for the safe computation of more digits. On the other hand, an attempt to compute atan(1) directly using the Taylor series would involve multiplying by $\log_{10} 1 = 0$ — thus we would not be assured of being able to accurately compute *any* digits of the result before encountering integer overflow in some intermediate calculation; this is one of the reasons we seek and use a mathematically equivalent calculation which is more numerically stable (the other reason is that even if we had infinite-precision variables, the rate of convergence for a raw atan(1) is much too slow).